

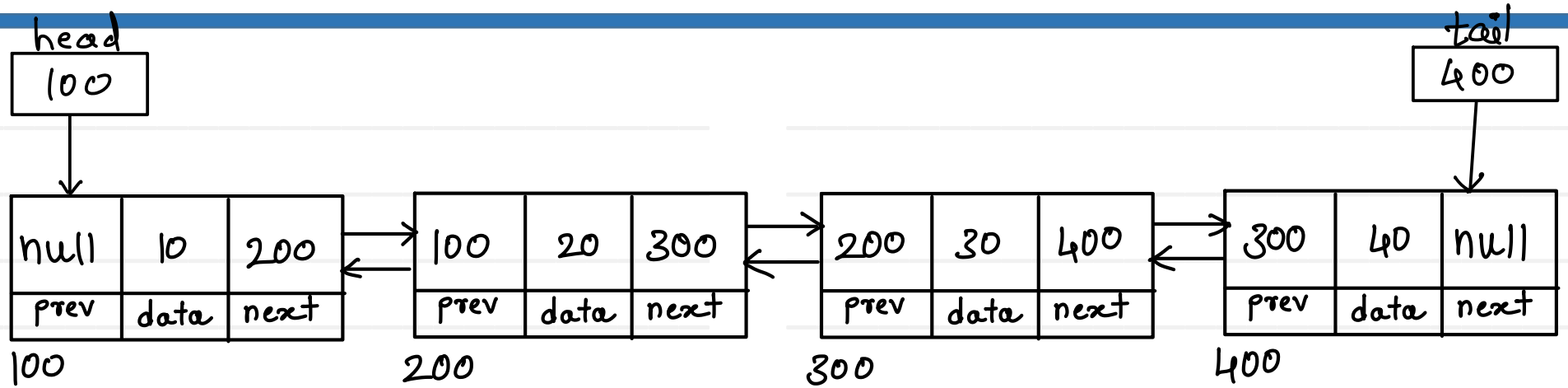
Sunbeam Institute of Information Technology
Pune and Karad

Algorithms and Data structures

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

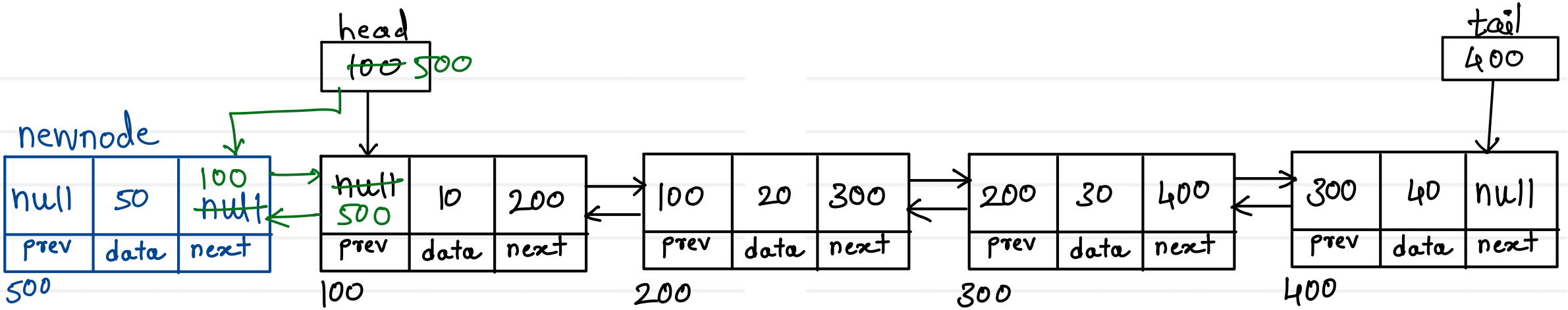
Doubly Linear Linked List - Display

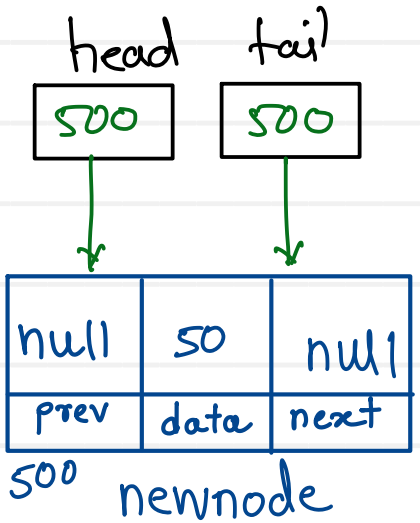


1. Create trav & start at first node
2. print current node data
3. go on next node
4. repeat step 2 & 3 till last node

1. Create trav & start at last node
2. print current node
3. go on prev node
4. repeat step 2 & 3 till first node

Doubly Linear Linked List - Add first

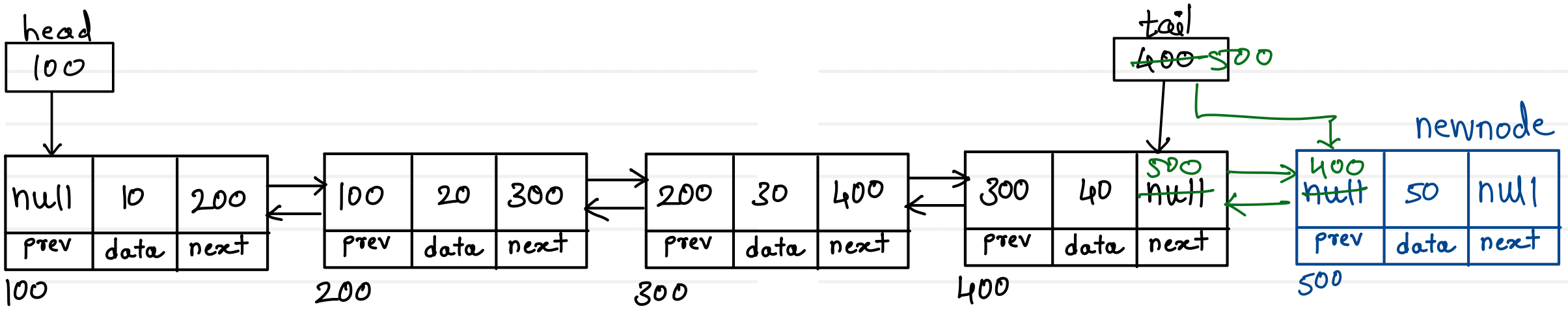


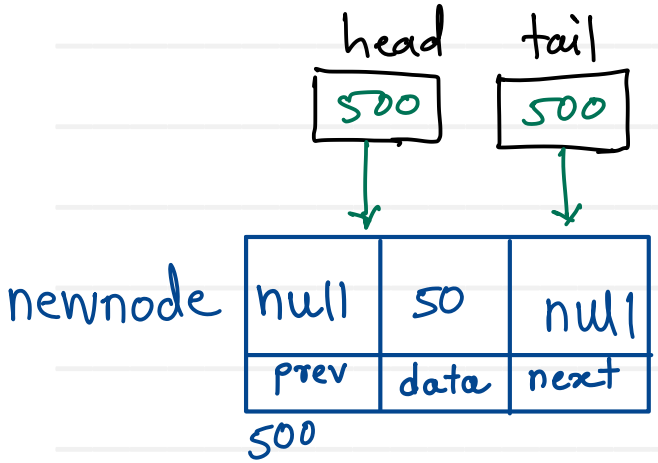


1. create a newnode
2. if list is empty, add newnode into head & tail
3. if list is not empty,
   a. add first node into next of newnode
   b. add newnode into prev of first node
   c. move head on newnode

$T(n) = O(1)$

Doubly Linear Linked List - Add last



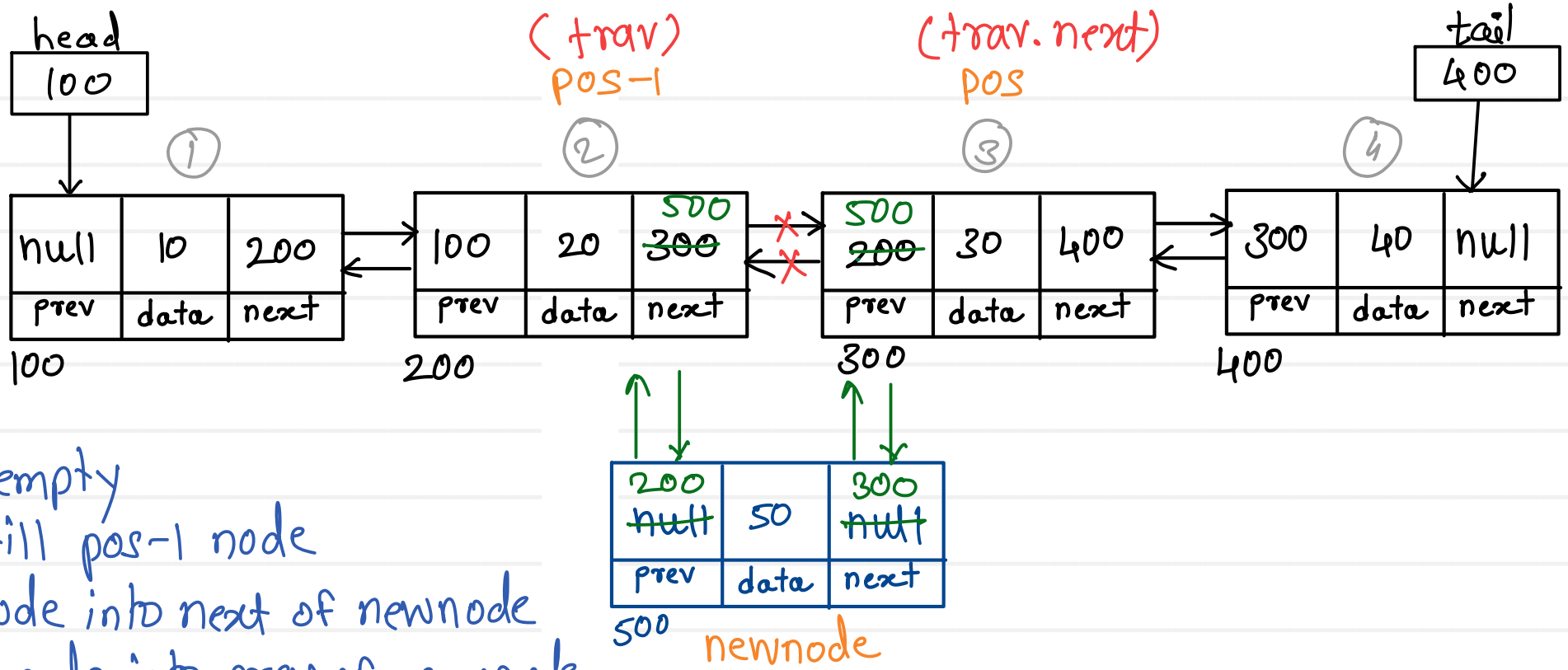


1. create a newnode
2. if list is empty, add newnode into head & tail
3. if list is not empty,
   a. add last node into prev of newnode
   b. add newnode into next of last node
   c. move tail on newnode

$T(n) = O(1)$

Doubly Linear Linked List - Add position

pos = 3

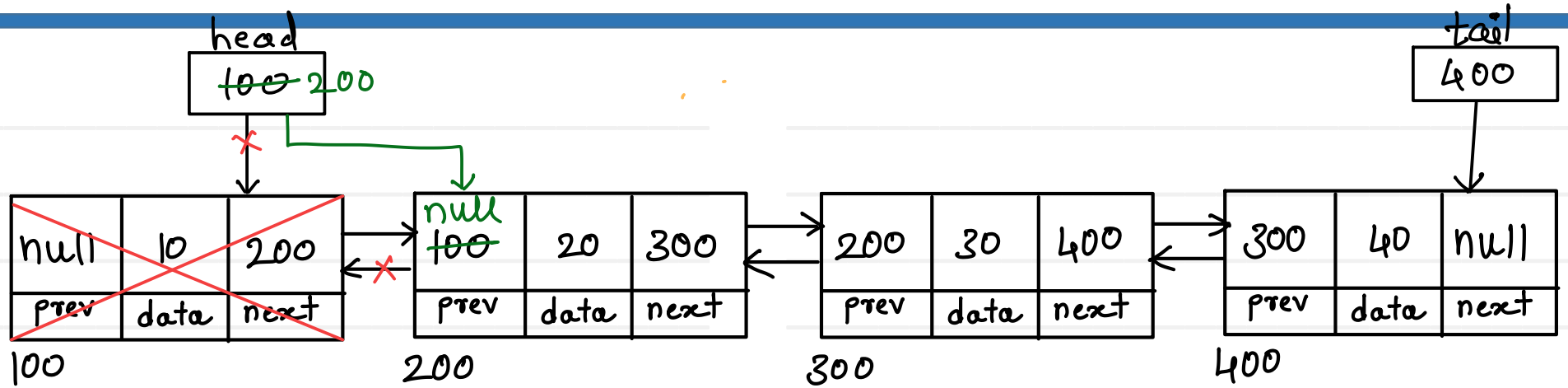


if list is not empty

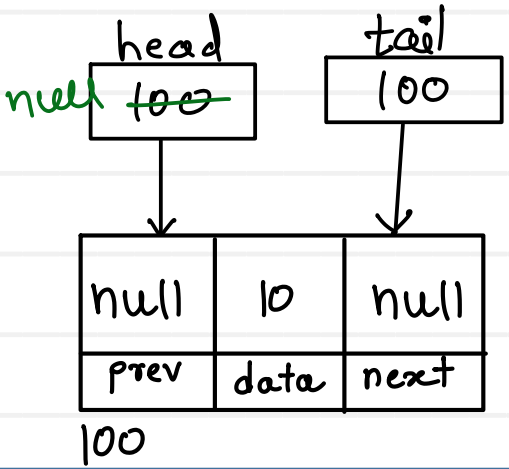
a. traverse till pos-1 node
b. add pos node into next of newnode
c. add pos-1 node into prev of newnode
d. add newnode into prev of pos node
e. add newnode into next of pos-1 node

$T(n) = O(n)$

Doubly Linear Linked List - Delete first



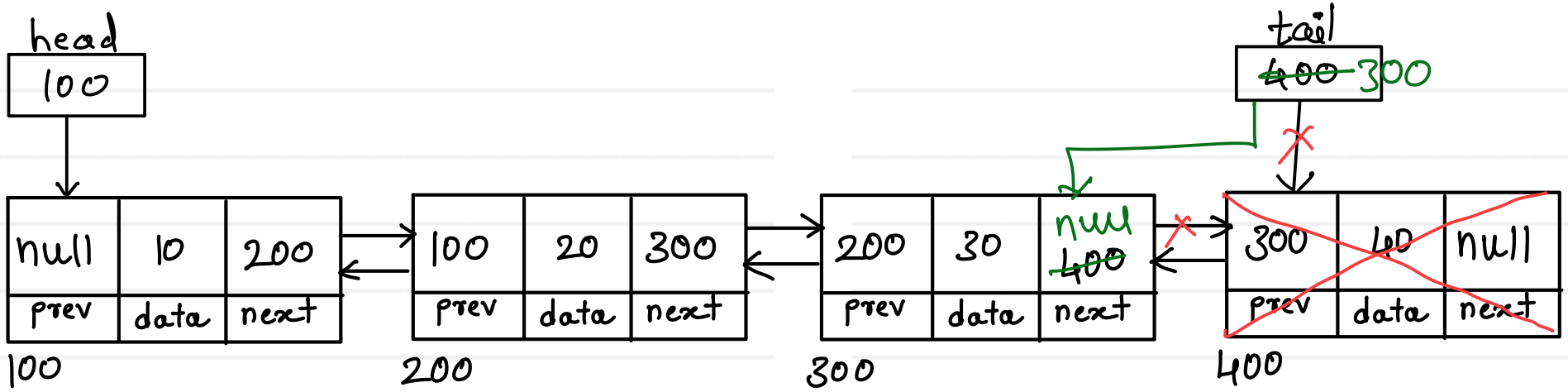
1. if list is empty, return
2. if list has single node, delete it
3. if list has multiple nodes
   a. move head on second node
   b. add null into prev of second node



head = head.next
head.prev = null
└ null pointer exception

T(n) = O(1)

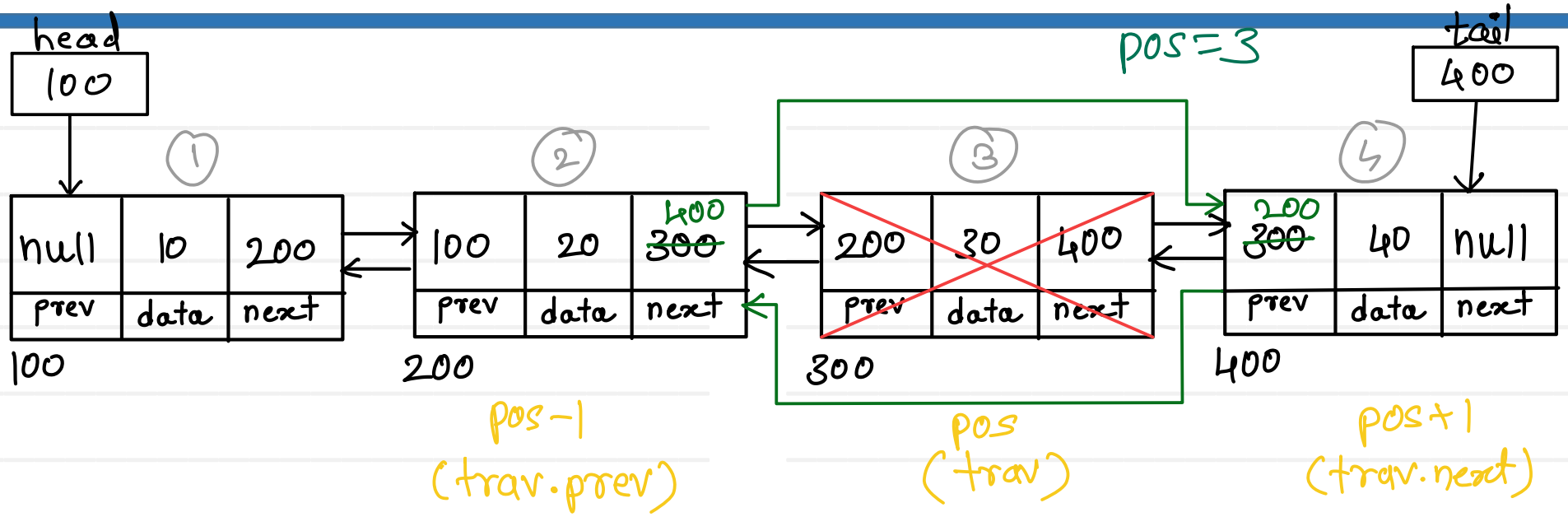
Doubly Linear Linked List - Delete last



1. if list is empty, return
2. if list has single node, delete it
3. if list has multiple nodes
   a. move tail on second last node
   b. add null into next of second last node

$T(n) = O(1)$

Doubly Linear Linked List - Delete Position

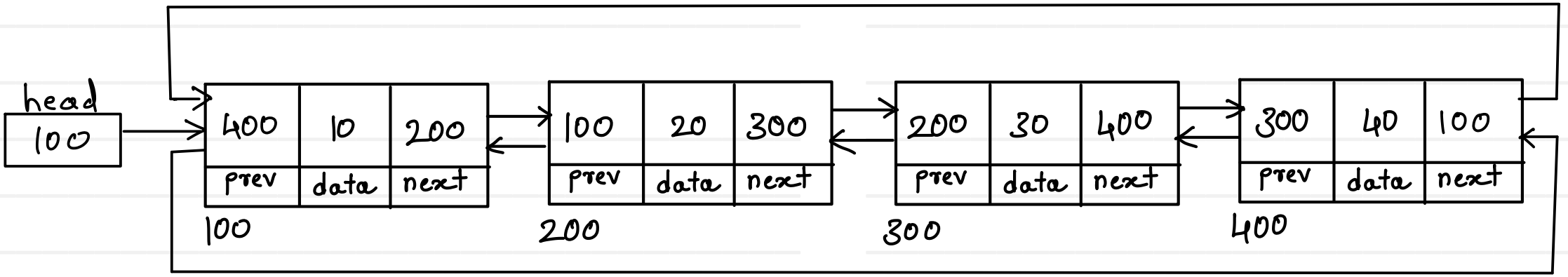


if list is not empty,

a. traverse till pos node
b. add pos-1 node into prev of pos+1 node
c. add pos+1 node into next of pos-1 node

$T(n) = O(n)$

Doubly Circular Linked List - Display

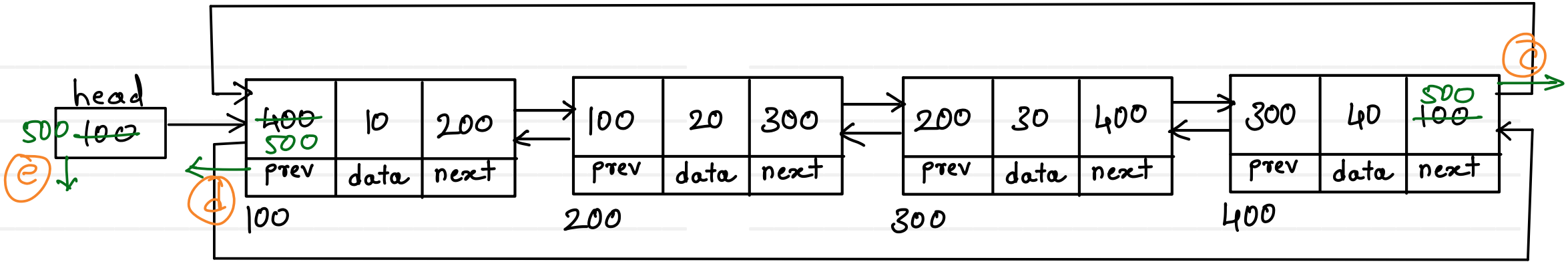


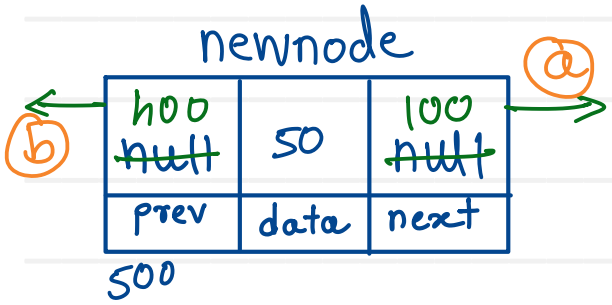
1. Create trav & start at first node
2. print current node data
3. go on next node
4. repeat step 2 & 3 till last node

1. Create trav & start at last node
2. print current node
3. go on prev node
4. repeat step 2 & 3 till first node

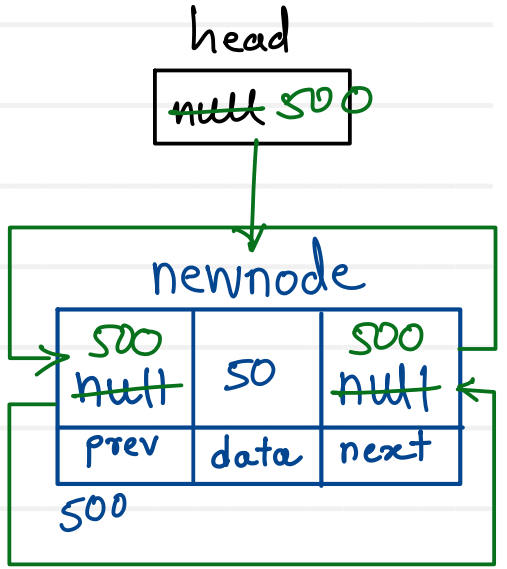
$$T(n) = O(n)$$

Doubly Circular Linked List - Add first



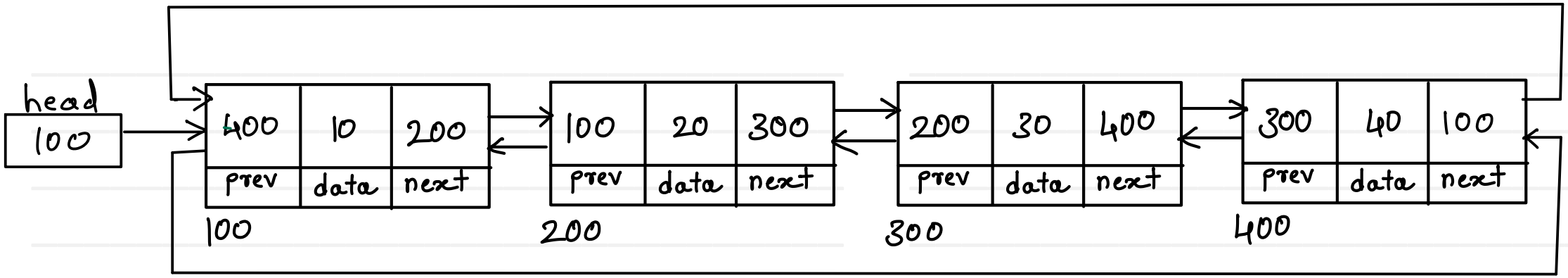


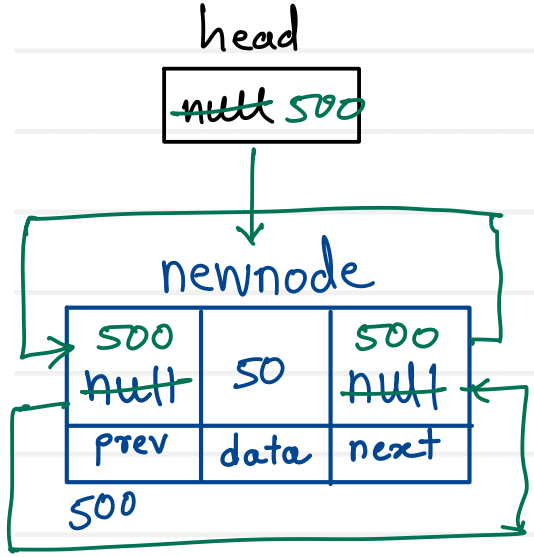
1. Create a newnode
2. if list is empty,
   a. add newnode into head
   b. make list circular
3. if list is not empty,
   a. add first node into next of newnode
   b. add last node into prev of newnode
   c. add newnode into next of last node
   d. add newnode into prev of first node
   e. move head on newnode.



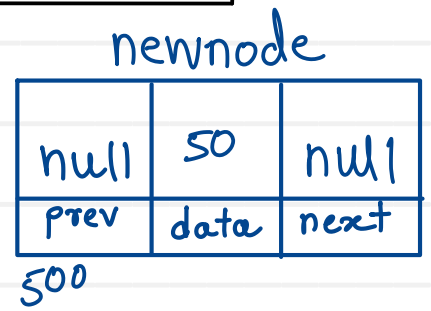
$T(n) = O(1)$

Doubly Circular Linked List - Add last



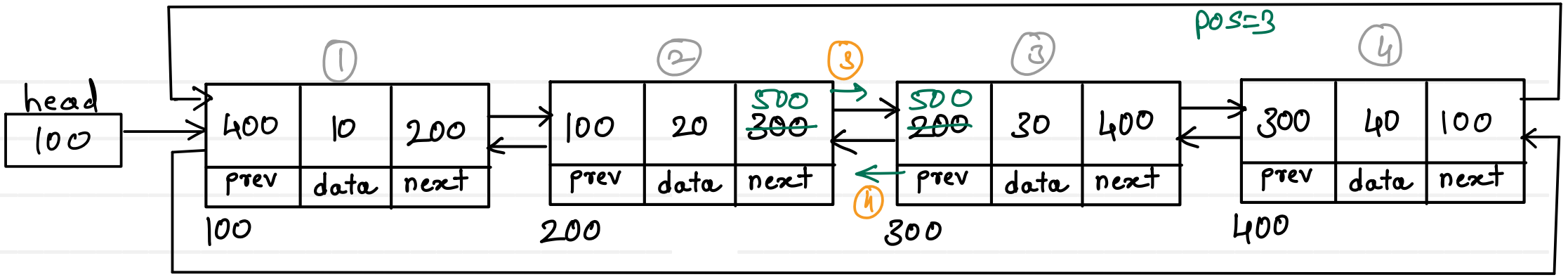


1. Create a newnode
2. if list is empty,
   a. add newnode into head
   b. make list circular
3. if list is not empty,
   a. add first node into next of newnode
   b. add last node into prev of newnode
   c. add newnode into next of last node
   d. add newnode into prev of first node

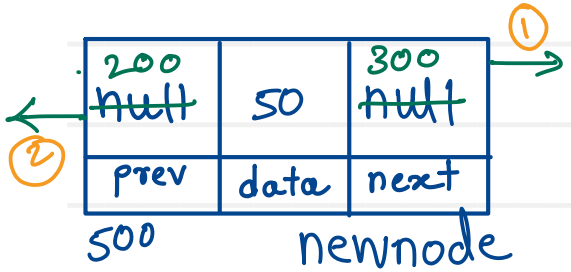


$T(n) = O(1)$

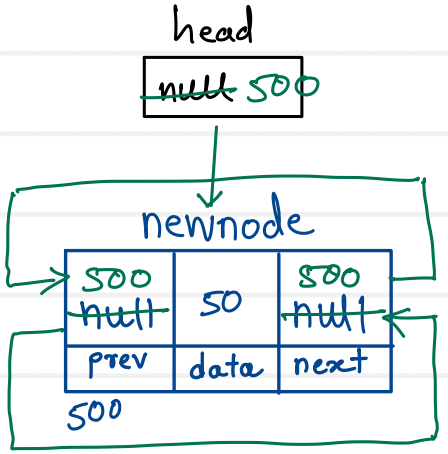
Doubly Circular Linked List - Add position



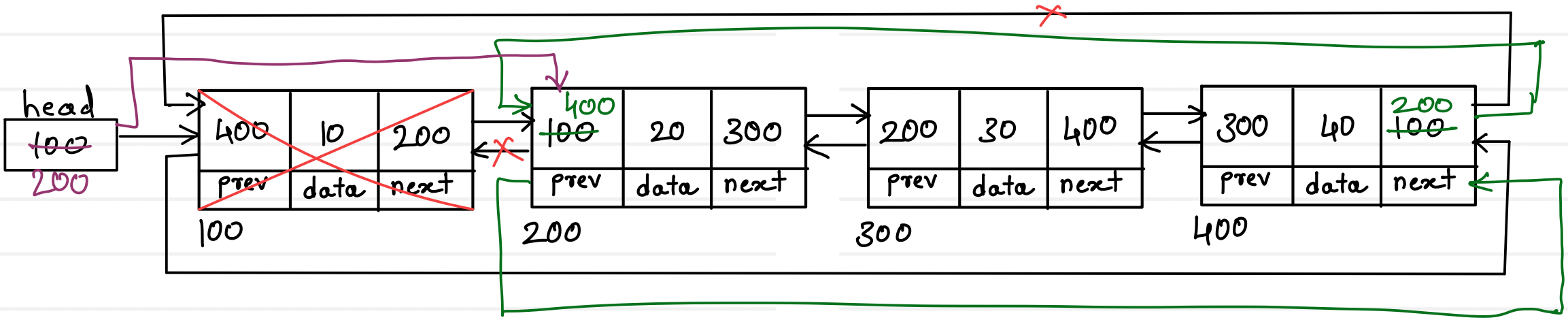
1. create node
2. if list is empty
   a. add newnode into head
   b. make list circular
3. if list is not empty.
   a. traverse till pos-1 node
   b. add pos node into next of newnode
   c. add pos-1 node into prev of newnode
   d. add newnode into next of pos-1 node
   e. add newnode into prev of pos node

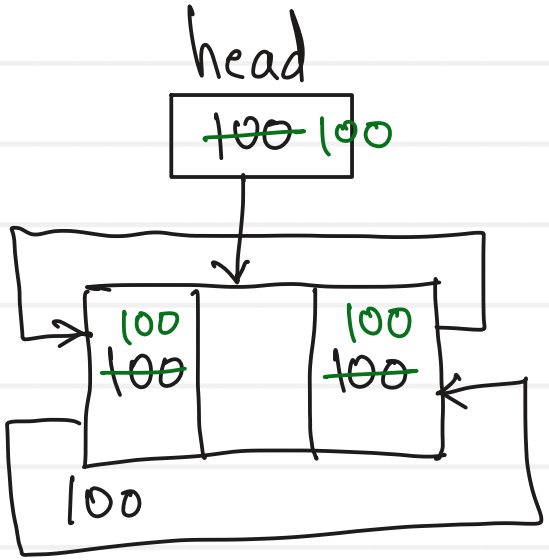


$T(n) = O(1)$



Doubly Circular Linked List - Delete first



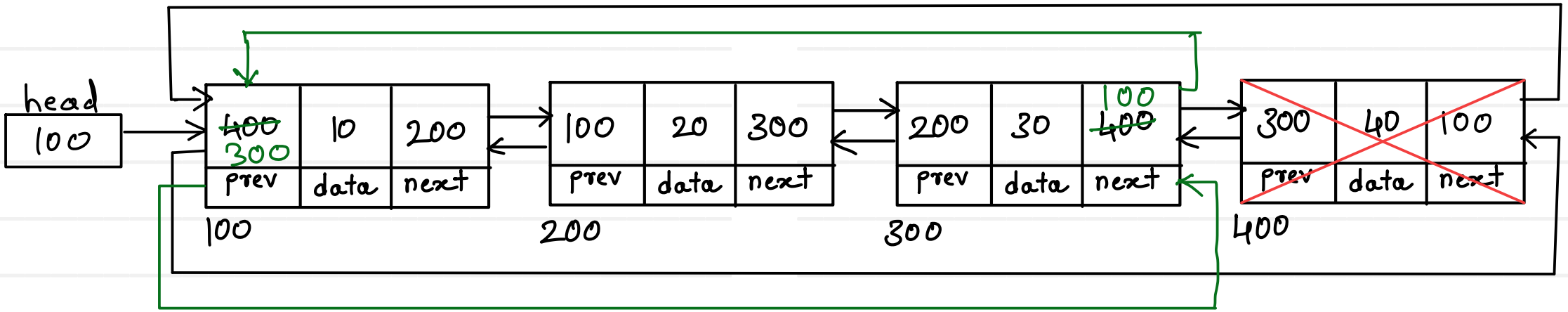



```
head.prev.next = head.next
head.next.prev = head.prev
head = head.next;
```

1. if empty, return
2. if single node, delete it
3. if multiple nodes,
   a. add second node into next of last node
   b. add last node into prev of second node
   c. move head on second node

$T(n) = O(1)$

Doubly Circular Linked List - Delete last

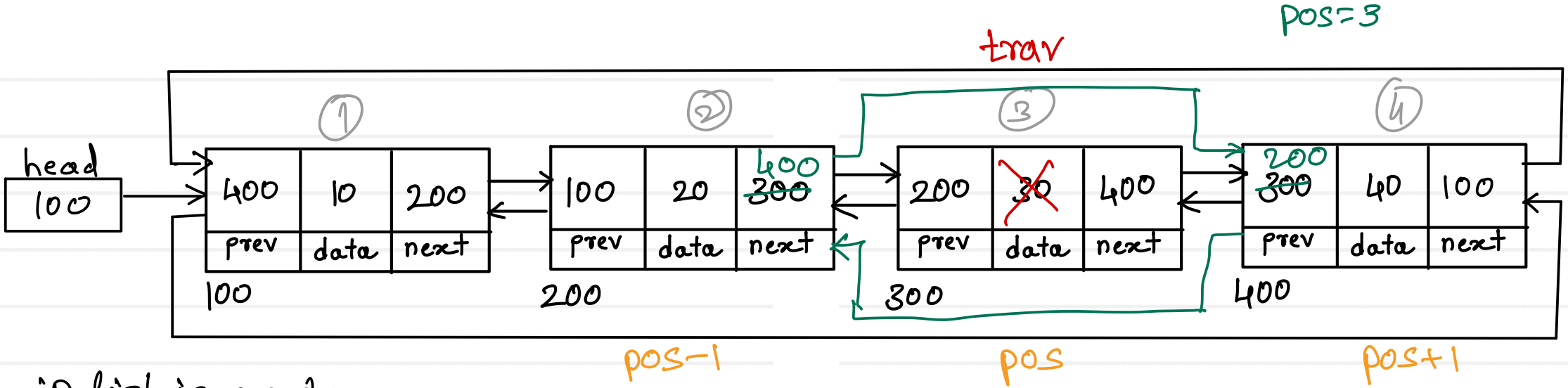


if. not empty,

1. add first node into next of second last node
2. add second last node into prev of first node

$T(n) = O(1)$

Doubly Circular Linked List - Delete position



1. if list is empty
   return;
2. if list has single node
   head = tail = null;
3. if list has multiple nodes
   a. traverse till pos node
   b. add pos+1 node into next of pos-1 node
   c. add pos-1 node into prev of pos+1 node

trav -> pos
trav.prev -> pos-1
trav.next -> pos+1

$T(n) = O(n)$

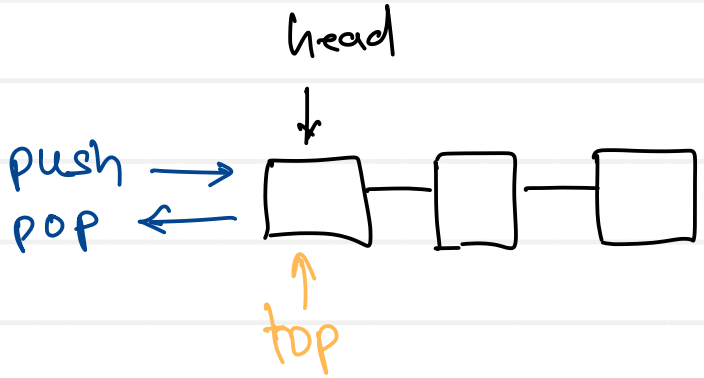
Stack and Queue using linked list

Stack

- single end : top
- LIFO

1. Add/Delete first
2. Add/Delete last

Time complexity of operations = O(1)

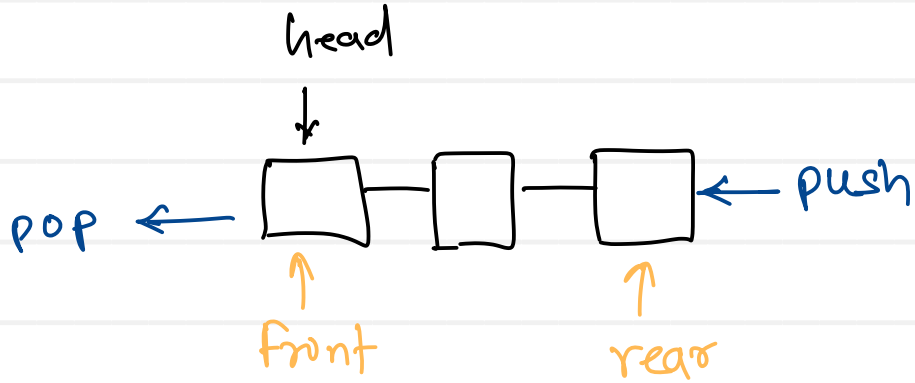


Queue

- two ends : front, rear
- FIFO

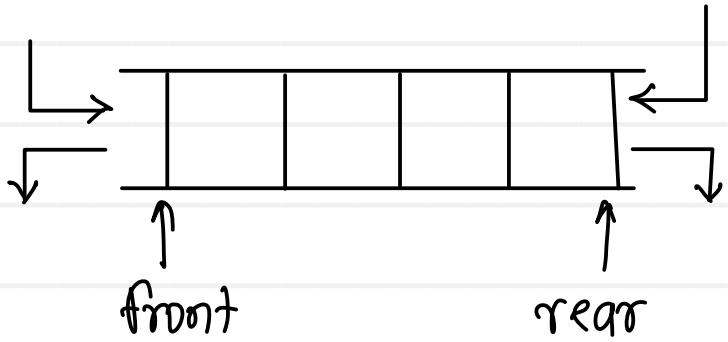
1. Add first, Delete last
2. Add last, Delete first

Time complexity of operations = O(1)



Double Ended Queue - Deque

- data can be inserted or removed in a queue from both ends



- Deque do not follow "FIFO" principle

- to implement double ended queue, we can use circular array or linked list

Operations :

1. pushFront()
2. popFront()
3. peekFront()
4. pushRear()
5. popRear()
6. peekRear()

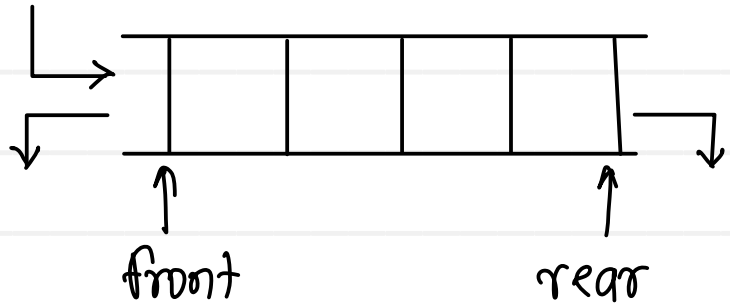
7. isEmpty()
8. isFull()

Types of Deque

Input Restricted Deque

remove from both ends

insert from only one end

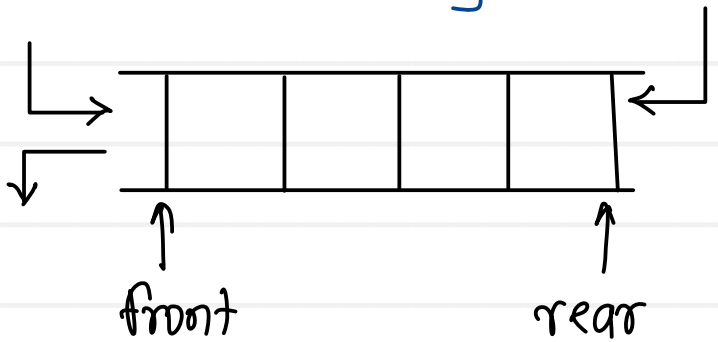


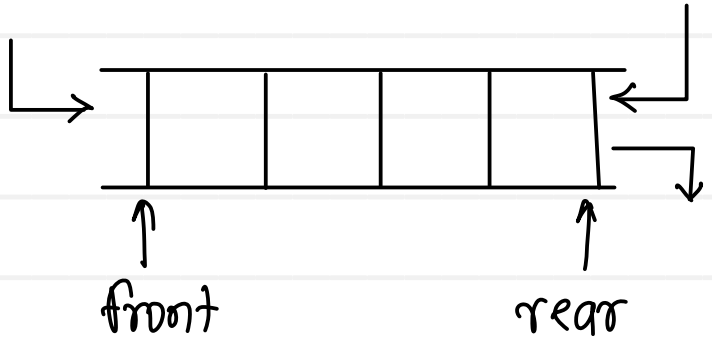


Output Restricted Deque

insert from both ends

remove from only one end

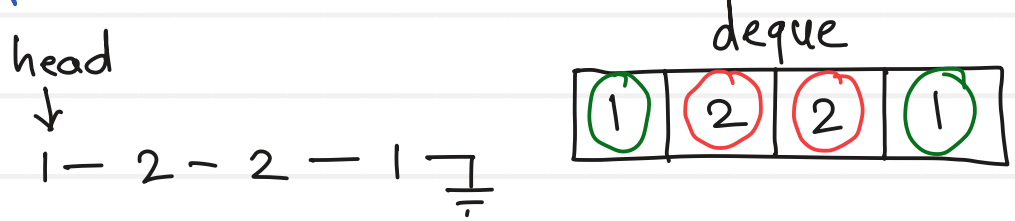




Applications of Deque

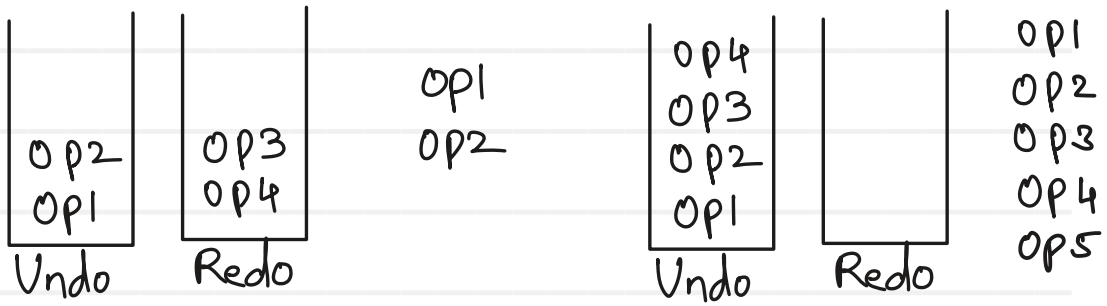
1. stack and queue are efficiently implemented using deque.

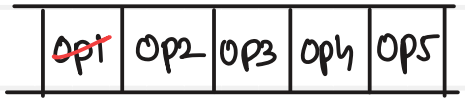
2. deque can be used to check list is palindrome or not



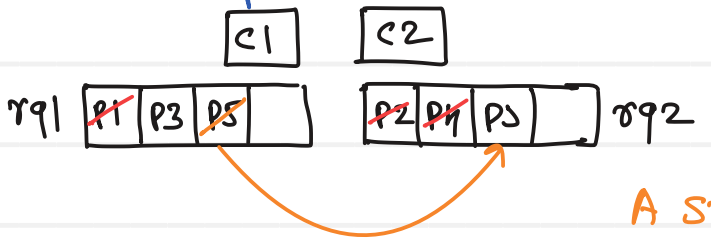
3. browser history is also managed with deque (forward & backward button)

4. Undo & Redo operations are done efficiently with deque





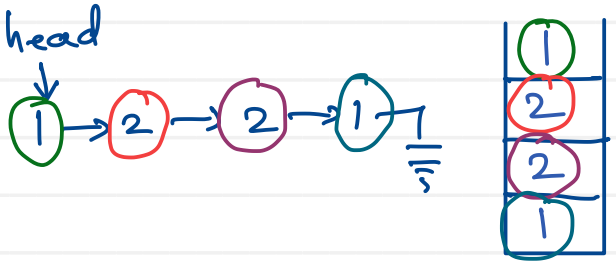
5. deque is used for process scheduling in multiprocessor environment



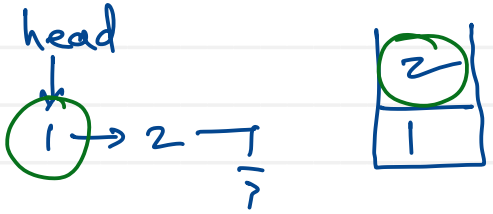
Palindrome Linked List

Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

Example 1:
Input: head = [1,2,2,1]
Output: true



Example 2:
Input: head = [1,2]
Output: false



to push – n
to compare – n
$T \propto 2n$

$T(n) = O(n)$

stack size = n
$S \propto n$

$S(n) = O(n)$

```
boolean isPalindrome(Node head) {
    Stack<Integer> st = new Stack<>();
    Node trav = head;
    while (trav != null) {
        st.push(trav.data);
        trav = trav.next;
    }
    Node trav = head;
    while(!st.isEmpty()) {
        if(trav.data != st.pop())
            return false;
        trav = trav.next;
    }
    return true;
}
```

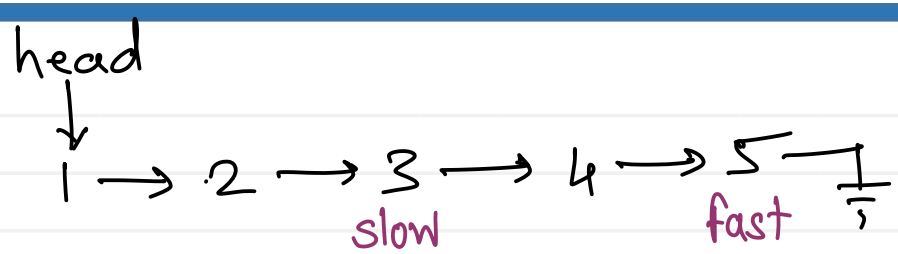
Middle of the Linked List

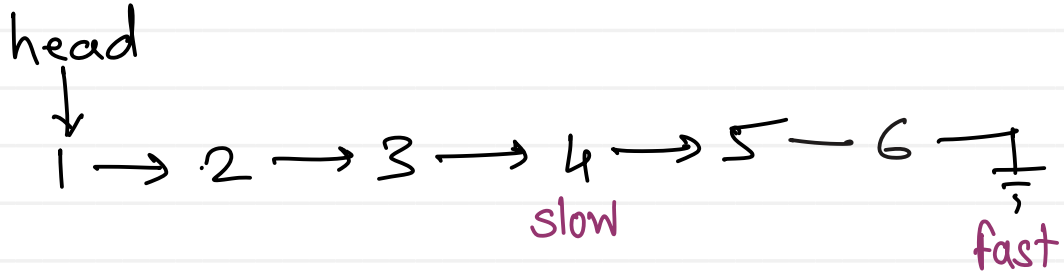
Given the head of a singly linked list, return the middle node of the linked list.

If there are two middle nodes, return the second middle node.

Example 1:
Input: head = [1,2,3,4,5]
Output: [3,4,5]
Explanation: The middle node of the list is node 3.

Example 2:
Input: head = [1,2,3,4,5,6]
Output: [4,5,6]
Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.






```
fast = slow = head;
while( fast != null && fast.next != null) {
    fast = fast.next.next;
    slow = slow.next;
}
sysout (slow.data);
```

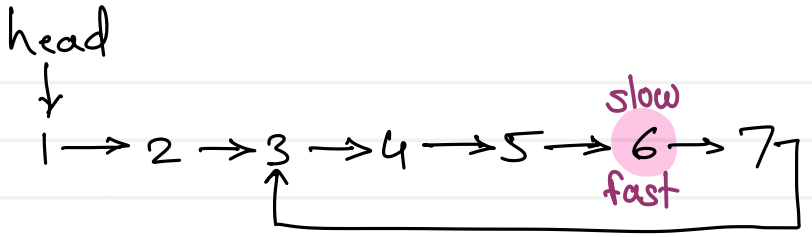
Linked List Cycle

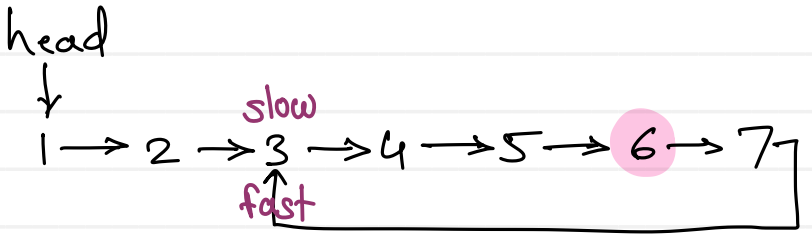
(Floyd Cyclic Detection)

Given head, the head of a linked list, determine if the linked list has a cycle in it.

Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter.

Return true if there is a cycle in the linked list. Otherwise, return false.





Example 1:
Input: head = [3,2,0,-4], pos = 1
Output: true

Example 2:
Input: head = [1,2], pos = 0
Output: true

Example 3:
Input: head = [1], pos = -1
Output: false

```
fast = slow = head;
while( fast != null && fast.next != null) {
    fast = fast.next.next;
    slow = slow.next;
    if(fast == slow)
        return true;
}
return false;
```

Two Sum II - Input Array Is Sorted

- Given a 1-indexed array of integers numbers that is already sorted in non-decreasing order, find two numbers such that they add up to a specific target number. Let these two numbers be numbers[index1] and numbers[index2] where 1 <= index1 < index2 <= numbers.length.
- Return the indices of the two numbers index1 and index2, each incremented by one, as an integer array [index1, index2] of length 2.
- The tests are generated such that there is exactly one solution. You may not use the same element twice.
- Your solution must use only constant extra space.

- Example 1:
- Input: numbers = [2,7,11,15], target = 9
- Output: [1,2]

i j

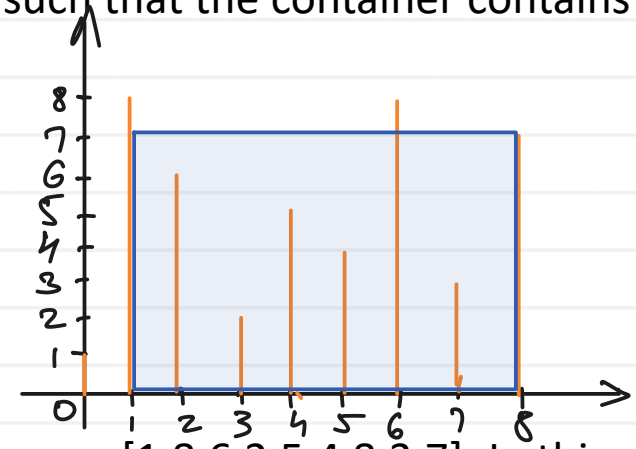
i	j	sum	
0	3	17	>
0	2	13	>
0	1	9	=

- Example 2:
- Input: numbers = [2,3,4], target = 6
- Output: [1,3]

- Example 3:
- Input: numbers = [-1,0], target = -1
- Output: [1,2]

Container With Most Water

- You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the ith line are (i, 0) and (i, height[i]).
- Find two lines that together with the x-axis form a container, such that the container contains the most water.
- Return the maximum amount of water a container can store.
- Notice that you may not slant the container.



- Example 1:
  - Input: height = [1,8,6,2,5,4,8,3,7]
  - Output: 49
  - Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.

- Example 2:
  - Input: height = [1,1]
  - Output: 1

i	j	height	width	area	maxArea
0	8	1	8	8	~~8~~
1	8	7	7	49	49
1	7	3	6	18	